

ASSIGNMENT 4

1. Differentiate between StringBuffer and StringBuilder in Java

StringBuffer and StringBuilder are used to create mutable (modifiable) strings in Java.

Key Differences:

- StringBuffer is thread-safe (synchronized), while StringBuilder is not thread-safe.
- StringBuffer is slower, while StringBuilder is faster.
- StringBuffer is used in multi-threaded applications, while StringBuilder is used in single-threaded applications.

Example:

```
StringBuffer sb1 = new StringBuffer("Hello");  
sb1.append(" Java");
```

```
StringBuilder sb2 = new StringBuilder("Hello");  
sb2.append(" World");
```

2. Java Program for Type Casting

```
public class TypeCastingDemo {  
    public static void main(String[] args) {  
  
        // Implicit Casting  
        int num = 100;  
        double d = num;  
  
        System.out.println("Implicit Casting:");  
        System.out.println("Integer value: " + num);  
        System.out.println("Double value: " + d);  
  
        // Explicit Casting  
        double price = 99.99;  
        int p = (int) price;  
  
        System.out.println("\nExplicit Casting:");  
        System.out.println("Double value: " + price);  
        System.out.println("Integer value: " + p);  
  
        // Char to int  
        char ch = 'A';
```

```

        int ascii = ch;

        System.out.println("\nCharacter Casting:");
        System.out.println("Character: " + ch);
        System.out.println("ASCII value: " + ascii);
    }
}

```

3. Wrapper Classes in Java

Wrapper classes convert primitive data types into objects.

Examples:

- int → Integer
- double → Double
- char → Character
- boolean → Boolean

Java Program:

```

public class WrapperClassDemo {
    public static void main(String[] args) {

        // Primitive to Integer object (Autoboxing)
        int num = 50;
        Integer obj = num;

        // Integer object to primitive (Unboxing)
        int value = obj;

        System.out.println("Primitive int value: " + num);
        System.out.println("Integer object value: " + obj);
        System.out.println("Converted back to primitive: " + value);
    }
}

```

4. Difference Between Static and Non-Static Method

```
class Demo {

    // Static method
    static void staticMethod() {
        System.out.println("This is a static method");
    }

    // Non-static method
    void nonStaticMethod() {
        System.out.println("This is a non-static method");
    }
}

public class MethodDemo {
    public static void main(String[] args) {

        // Calling static method
        Demo.staticMethod();

        // Calling non-static method
        Demo obj = new Demo();
        obj.nonStaticMethod();
    }
}
```

5. Inheritance in OOP

Inheritance is a feature in Object-Oriented Programming where one class acquires the properties and methods of another class.

- Parent class → Superclass
- Child class → Subclass

Single Level Inheritance means one child class inherits from one parent class.

Java Program:

```
// Parent class
class Animal {
```

```
        void sound() {  
            System.out.println("Animal makes a sound");  
        }  
    }
```

// Child class

```
class Dog extends Animal {  
  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

```
public class InheritanceDemo {  
    public static void main(String[] args) {  
  
        Dog obj = new Dog();  
  
        obj.sound(); // inherited method  
        obj.bark();  // child class method  
    }  
}
```